

Arrays and Hashing: The Bread and Butter of LeetCode

Mastering Core Mechanics, Patterns, and Optimal Problem Solving

1. Fundamental Concepts

- **Dynamic Arrays:** Resizable contiguous memory blocks.
 - Access via index is $O(1)$.
 - Insertions/deletions at the beginning or middle take $O(n)$ (shifting elements).
- **Hashing:** Uses a hash function to map data to a specific array index.
 - **Average Case:** $O(1)$ for search, insert, and delete.
 - **Collisions:** Handled via Chaining (linked lists) or Open Addressing (finding next empty slot).
 - **Load Factor:** Ratio of elements to buckets. Triggers resizing/rehashing when too high.

2. Essential Problem-Solving Patterns

Pattern	Logic	Example Problems
Complement Lookup	Store "seen" values to find pairs in one pass.	Two Sum
Frequency Counting	Track element occurrences.	Valid Anagram, Top K
Prefix Sums	Precompute cumulative sums for range queries.	Subarray Sum Equals K
In-Place Marking	Use array indices as "keys" (negate values).	Find All Duplicates
Grouping by Key	Transform elements into canonical form as map keys.	Group Anagrams

3. Optimization & Best Practices

- **Fixed-Size Arrays vs. Maps:** For small, known input ranges (e.g., 26 lowercase letters), `int[26]` is faster and more space-efficient than a full HashMap.
- **Immutable Keys:** In Python, only immutable types (tuples, strings, integers) can be dictionary keys. Convert lists to tuples to hash them!
- **Set for Uniqueness:** If you don't need index/frequency, a `HashSet` is the most efficient choice.
- **Sorting as Preprocessing:** Sometimes $O(n \log n)$ sorting simplifies the problem to a two-pointer approach, avoiding complex hashing entirely.

Contains Duplicate

Problem: Given an integer array `nums`, return `true` if any value appears at least twice; return `false` if every element is distinct.

- **Brute-Force:** Nested loops covering every pair. $O(n^2)$ time, $O(1)$ space.
- **Optimal (Hash Set):** Use a Set to track seen numbers. $O(n)$ time, $O(n)$ space.

Contains Duplicate: Core Concepts (Sets)

Java (`HashSet<Integer>`)

- Implements `Set` interface storing unique elements.
- `set.add(element)` : Returns `false` if duplicate exists, `true` otherwise.
- `set.contains(element)` : $O(1)$ lookup.

Python (`set`)

- Built-in unordered collection of unique elements.
- `my_set.add(element)` : Adds element. Duplicate additions do nothing.
- `element in my_set` : $O(1)$ lookup.
- `set(nums)` : Instantly removes all duplicates from a list.

Contains Duplicate: Optimal Approach (Early Exit)

Concept: Iterate once, keeping a Set of seen numbers. Stop as soon as you find a duplicate.

Time & Space Complexity

- **Time:** $O(n)$ (Array traversed once, Set queries are $O(1)$)
- **Space:** $O(n)$ (Set may store up to n unique elements)

Contains Duplicate: Algorithm (Pseudocode)

```
FUNCTION hasDuplicate(array nums):  
    CREATE an empty Hash Set called 'seen'  
  
    FOR EACH number 'num' IN nums:  
        IF 'num' EXISTS IN 'seen':  
            RETURN True    // Duplicate found!  
  
        ADD 'num' TO 'seen'  
  
    RETURN False           // All elements are unique
```


Contains Duplicate: Early Exit (Java)

```
import java.util.HashSet;
import java.util.Set;

class Solution {
    public boolean hasDuplicate(int[] nums) {
        // Initialize an empty HashSet to store numbers we've seen
        Set<Integer> seenNumbers = new HashSet<>();

        // Loop through each number in the array
        for (int num : nums) {
            // If the set already contains the number, we found a duplicate
            if (seenNumbers.contains(num)) {
                return true; // Found duplicate
            }
            // Otherwise, add the number to the set and continue tracking
            seenNumbers.add(num);
        }
        // If the loop finishes without returning, all elements are unique
        return false;
    }
}
```

Contains Duplicate: Early Exit (Python)

```
class Solution:
    def hasDuplicate(self, nums: list[int]) -> bool:
        # Initialize an empty set to store numbers we've seen
        seen_numbers = set()

        # Loop through each number in the list
        for num in nums:
            # If the number is already in the set, we found a duplicate
            if num in seen_numbers:
                return True # Found duplicate
            # Otherwise, add the number to the set
            seen_numbers.add(num)

        # If the loop finishes without returning, all elements are unique
        return False
```

Contains Duplicate: Set Size Comparison

Concept: Convert the array to a Set. Sets automatically remove duplicates. Compare their sizes.

- If sizes differ, duplicates were present!

Python implementation

```
class Solution:
    def hasDuplicate(self, nums: list[int]) -> bool:
        # Convert the list to a set (automatically removes all duplicates)
        unique_nums = set(nums)

        # Compare sizes: if sizes differ, duplicates were present and removed
        return len(unique_nums) != len(nums)
```

- *Extremely elegant and Pythonic. $O(n)$ time, $O(n)$ space.*

Valid Anagram

Problem: Given two strings **s** and **t**, determine if **t** is an anagram of **s**.

(Anagrams have exact same characters with exact same frequencies, but order can differ).

- **Brute-Force:** Sort both strings and compare.
 - Time: $O(n \log n)$ | Space: $O(n)$.
- **Optimal:** Count character frequencies.
 - Time: $O(n)$ | Space: $O(1)$ (using fixed array for English alphabet).

Valid Anagram: Core Concepts

Java

- `s.charAt(i)` : Access characters.
- `ch - 'a'` : Converts lowercase char to array index (0 — 25).
- `new int[26]` : Fixed-size array for tracking counts.

Python

- `ord(char) - ord('a')` : Converts char to ASCII index.
- `counts = [0] * 26` : Fixed-size frequency array.
- `collections.Counter(s)` : Instantly builds a frequency map.

Valid Anagram: Algorithm (Pseudocode)

```
FUNCTION isAnagram(string s, string t):  
    IF length of s != length of t:  
        RETURN False  
  
    // Create an Array of size 26 initialized with 0s  
    CREATE array 'counts' of size 26  
  
    FOR EACH character 'char' IN s:  
        index = ASCII_VALUE(char) - ASCII_VALUE('a')  
        INCREMENT counts[index] BY 1  
  
    FOR EACH character 'char' IN t:  
        index = ASCII_VALUE(char) - ASCII_VALUE('a')  
        DECREMENT counts[index] BY 1  
  
        IF counts[index] < 0:  
            RETURN False    // 't' has more of this char than 's'  
  
    RETURN True
```

Valid Anagram: Optimal Array Approach (Java)

Use a fixed size `int[26]` to track character balance. Increment for `s`, decrement for `t`.

```
class Solution {
    public boolean isAnagram(String s, String t) {
        // Anagrams must be the exact same length
        if (s.length() != t.length()) return false;

        // Initialize an array of size 26 for counting lowercase English letters
        int[] counts = new int[26];

        // Increment the count for each character present in string 's'
        for (int i = 0; i < s.length(); i++) {
            counts[s.charAt(i) - 'a']++;
        }

        // Decrement the count for each character in string 't'
        for (int i = 0; i < t.length(); i++) {
            counts[t.charAt(i) - 'a']--;

            // If any count falls below 0, 't' has more of this character than 's'
            if (counts[t.charAt(i) - 'a'] < 0) return false;
        }

        // If no counts fell below zero (and lengths match), they are anagrams
        return true;
    }
}
```

Valid Anagram: Optimal Array Approach (Python)

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        # Anagrams must be the exact same length
        if len(s) != len(t): return False

        # Initialize an array of 26 zeros to track character frequencies
        counts = [0] * 26

        # Increment the frequency count for each character in string 's'
        for char in s:
            counts[ord(char) - ord('a')] += 1

        # Decrement the frequency count for each character in string 't'
        for char in t:
            idx = ord(char) - ord('a')
            counts[idx] -= 1

            # If any count falls below 0, 't' has more of this character than 's'
            if counts[idx] < 0: return False

        # If no counts fell below zero, they are anagrams
        return True
```


Valid Anagram: Counter Approach (Python specific)

If working with general Unicode or unknown alphabet sizes, a standard Hash Map or `Counter` is safest.

```
from collections import Counter

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        # Quick length check
        if len(s) != len(t):
            return False

        # Counter instantly creates a Hash Map of {character: frequency count}
        # We can then compare the two Counter maps directly
        return Counter(s) == Counter(t)
```

- *Time Complexity remains $O(n)$, space scales to $O(k)$ distinct chars.*

Two Sum

Problem: Given integer array `nums` and target, return indices of the two numbers that add up to target. (Guaranteed exactly one valid answer).

- **Brute-Force:** Nested loop checking every pair i, j . $O(n^2)$ time.
- **Optimal (One-Pass Mapping):** Use Hash Map to map `number -> index` while searching for complement. $O(n)$ time.

Two Sum: One-Pass Hash Map Idea

Logic: As you traverse the array, for the current `num` :

1. Calculate `complement = target - num`.
2. Check if `complement` exists in the Hash Map.
 - **If yes:** You've found the pair! Return their indices.
 - **If no:** Store `num` and its index in the Hash Map for future reference.

Two Sum: Algorithm (Pseudocode)

```
FUNCTION twoSum(array nums, integer target):  
    CREATE an empty Hash Map called 'map' // stores {number: index}  
  
    FOR i = 0 TO nums.length - 1:  
        current_num = nums[i]  
        complement = target - current_num  
  
        IF complement EXISTS AS KEY IN 'map':  
            RETURN [ map.get(complement), i ]  
  
        // Otherwise, store current_num and its index  
        ADD (current_num, i) TO 'map'  
  
    RETURN [] // Only reached if no solution exists
```

Two Sum: Optimal Implementation (Java)

```
import java.util.HashMap;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        // Map to store {number -> its index}
        HashMap<Integer, Integer> map = new HashMap<>();

        // Iterate through the array once
        for (int i = 0; i < nums.length; i++) {
            // Find the number needed to reach the target sum
            int complement = target - nums[i];

            // If the map already has our complement, we found the pair!
            if (map.containsKey(complement)) {
                // Return the index of the complement and the current index
                return new int[]{ map.get(complement), i };
            }

            // Otherwise, store the current number and index for future reference
            map.put(nums[i], i);
        }
        // This won't be reached since EXACTLY ONE valid answer is guaranteed by the prompt
        return new int[]{};
    }
}
```

Two Sum: Optimal Implementation (Python)

```
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        # Dictionary to store {number: its index}
        num_map = {}

        # Iterate through the list, extracting both index (i) and value (num)
        for i, num in enumerate(nums):
            # Calculate the number needed to reach the target
            complement = target - num

            # If the needed number was seen previously, we found the matching pair!
            if complement in num_map:
                # Return the stored index of the complement and the current index 'i'
                return [num_map[complement], i]

            # Otherwise, store the current number and index for future reference
            num_map[num] = i

        return []
```

Group Anagrams

Problem: Group an array of strings such that all anagrams are contained in identical sub-lists.

Core Idea (Grouping by canonical key):

Anagrams have the exact same characters. If we **sort** an anagram alphabetically, it becomes a universal "key".

- **eat** sorted → **aet**
- **tea** sorted → **aet**

We use a Hash Map where the key is **aet** , and values are the original strings **["eat", "tea", "ate"]** .

Group Anagrams: Algorithm (Pseudocode)

```
FUNCTION groupAnagrams(array strs):  
    CREATE an empty Hash Map called 'groups'  
    // maps { canonical_string: list of words }  
  
    FOR EACH string 's' IN strs:  
        canonical_key = SORT_CHARACTERS(s)  
  
        IF canonical_key DOES NOT EXIST IN 'groups':  
            groups[canonical_key] = NEW empty list  
  
        APPEND 's' TO groups[canonical_key]  
  
    // Extract all group lists from the Hash Map  
    RETURN VALUES OF 'groups'
```


Group Anagrams: Optimal Sorting Approach

Complexity:

- **Time:** $O(N \cdot M \log M)$ (N strings of length M)
- **Space:** $O(N \cdot M)$

Python Implementation

```
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        # A dictionary where values default to an empty list
        groups = defaultdict(list)

        for s in strs:
            # Create a canonical key by alphabetically sorting the characters of the word
            sorted_key = "".join(sorted(s))
            # Append the original word to the list matching that canonical key
            groups[sorted_key].append(s)

        # Return all the grouped lists as a single list of lists
        return list(groups.values())
```

Group Anagrams: Java Implementation

```
import java.util.*;

class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        // Map canonical string keys to a list of matching anagram words
        Map<String, List<String>> groups = new HashMap<>();

        for (String str : strs) {
            // Create the canonical key by sorting the string's chars alphabetically
            char[] arr = str.toCharArray();
            Arrays.sort(arr);
            String key = new String(arr);

            // If the key doesn't exist yet, initialize a new empty list for it
            groups.putIfAbsent(key, new ArrayList<>());
            // Add the original string to the matching key's list
            groups.get(key).add(str);
        }
        // Convert the map's values (Lists of Strings) into an ArrayList of Lists
        return new ArrayList<>(groups.values());
    }
}
```

Top K Frequent Elements

Problem: Given integer array `nums` and integer `k`, return the `k` most frequent elements.

Core Steps:

1. **Count Frequencies:** Identify how many times each number appears using Hash Map.
2. **Select Top K:** Sort by frequency OR use Bucket Sort.

Top K Frequent Elements: Bucket Sort (Pseudocode)

```
FUNCTION topKFrequent(array nums, integer k):  
    CREATE a Hash Map 'freq_map' containing {number: count} for all nums  
  
    // Create an array of empty lists ("buckets")  
    CREATE array 'buckets' of size (nums.length + 1)  
  
    FOR EACH (num, count) IN freq_map:  
        APPEND 'num' TO buckets[count]  
  
    CREATE empty list 'result'  
  
    FOR i = buckets.length - 1 DOWN TO 1:  
        FOR EACH num IN buckets[i]:  
            APPEND 'num' TO 'result'  
            IF length of 'result' == k:  
                RETURN 'result'  
  
    RETURN 'result'
```

Top K Frequent Elements: Map + Sorting

1. Frequency mapping taking $O(n)$.
2. Extracting unique elements taking $O(u)$.
3. Sorting list by frequencies descending taking $O(u \log u)$.

Overall Time: $O(n \log n)$ in the worst case.

Python Map & Sort

```
from collections import Counter

class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        # Create a Hash Map of frequencies {number: count}
        freq_map = Counter(nums)

        # Get a list of all unique elements (from the dictionary keys)
        unique_elements = list(freq_map.keys())

        # Sort the unique elements list based on their frequency count (Descending)
        unique_elements.sort(key=lambda n: freq_map[n], reverse=True)

        # Return the first 'k' elements
        return unique_elements[:k]
```

Top K Frequent Elements: Map + Bucket Sort

Idea: Create buckets where the **index represents frequency**, containing a list of numbers that appear that many times.

- Since max frequency is `len(nums)`, we need `len(nums) + 1` buckets.
- Extremely efficient $O(n)$ time complexity!

Bucket Sort Implementation (Python)

```
from collections import Counter

class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        # Create a Hash Map of frequencies {number: count}
        counts = Counter(nums)

        # Create 'N + 1' buckets (an array of empty lists).
        # The exact index of the list will represent the frequency of items inside.
        buckets = [[] for _ in range(len(nums) + 1)]

        # Map each unique number into the bucket exactly matching its frequency
        for num, freq in counts.items():
            buckets[freq].append(num)

        res = []
        # Iterate backward starting from maximum possible frequency (len(nums))
        for i in range(len(nums), 0, -1):
            # For any numbers matching this frequency bucket, append to result
            for num in buckets[i]:
                res.append(num)
            # Early return once we've collected exactly 'k' elements
            if len(res) == k:
                return res

        return res
```

Bucket Sort Implementation (Java)

```
// Focus on main logic
Map<Integer, Integer> map = new HashMap<>();
// Populate a Map with the frequency of each number
for (int num : nums) map.put(num, map.getOrDefault(num, 0) + 1);

// Create an array of Lists functioning as 'buckets' where the index represents frequency
List<Integer>[] buckets = new List[nums.length + 1];
for (int i = 0; i < buckets.length; i++) buckets[i] = new ArrayList<>();

// For every number, place it into the bucket exactly matching its calculated frequency
for (int num : map.keySet()) buckets[map.get(num)].add(num);

int[] res = new int[k];
int index = 0;
// Iterate backwards from the highest possible frequency down to zero
for (int i = buckets.length - 1; i >= 0 && index < k; i--) {
    // Collect numbers from highest-frequency buckets first
    for (int num : buckets[i]) {
        res[index++] = num;
        // Optimization: Stop once exactly 'k' numbers are retrieved
        if (index == k) break;
    }
}
return res;
```


Conclusion

- **Hash Maps and Sets** are extremely powerful for optimizing brute-force solutions!
- **Sets** are perfect for detecting uniqueness (*Contains Duplicate*).
- **Arrays** mapping `char - 'a'` to index are optimized counting tools (*Anagrams*).
- **Maps** map elements to indices (*Two Sum*) or canonical keys (*Group Anagrams*).
- **Bucket Sort** utilizes bounded frequencies to solve *Top K* problems in linear time.